

A Hybrid Model Using Content-Based NCF and LETOR with XGBoost

Open-source platforms such as GitHub offer tremendous opportunities for developers to learn, build experience, and contribute to impactful software projects. However, the vast number of available repositories makes it difficult for developers to find suitable projects to contribute to. This challenge can lead to disengagement, and studies show that only around 2% of GitHub users become long-term contributors. A more effective recommendation system could help retain talented developers by directing them toward meaningful and engaging projects. This thesis presents a two-stage recommendation framework for GitHub project suggestions. To capture developer interest and prioritize relevant projects, we define the strength of user interest using seven types of interaction behaviors. The first stage focuses on candidate generation using a Neural Collaborative Filtering (NCF) model. To enrich project representation, we evaluated three project description embedding techniques: TF-IDF, BERTopic, and BERT, and applied clustering algorithms (K-Means, Hierarchical, DBSCAN, and HDBSCAN) to extract topic-level features from project descriptions. To further refine recommendations, we apply a Learning-to-Rank (LETOR) approach using an XGBoost model to re-rank the candidate list based on richer semantic, behavioral, and social features. Experimental results show that using BERT-based embeddings combined with DBSCAN clustering provides the highest quality topic features. In the one-stage recommendation setup, the best configuration reaches a Recall@10 of 0.08622. When combined with a LETOR using XGBoost, the two-stage pipeline significantly improves performance, reaching a final Recall@10 of 0.21193. These results demonstrate that our recommendation system effectively integrates semantic, behavioral, and social signals to deliver personalized OSS project suggestions.

DATA SOURCE

- GHTorrent
 - Training set: January 1, 2010– December 31, 2015
 - Validation set: January 1, 2016- December 31, 2018



RECOMMENDATION SYSTEM OVERVIEW

-
- The diagram illustrates a two-stage recommendation system architecture, divided into an Online path (top) and an Offline path (bottom) by a vertical line.
- Online Path (Top):**
- Request:** The process begins with a Request, which enters the **Retrieval** stage.
 - Retrieval Stage:** The Request is processed by the **Project corpus** (represented by a cylinder icon) to **Retrieve Top K Candidates** (represented by a cylinder icon).
 - Ranking Stage:** The Retrieved Top K Candidates are then processed by **Add feature to Candidates** (represented by a cylinder icon) and **Rank Top K Candidates** (represented by a cylinder icon).
 - Results:** The final output of the Ranking stage is the **Results**, represented by a row of colored circles.
- Offline Path (Bottom):**
- Training Data Preparation:** The Offline path starts with **Prepare Training Data** (represented by a cylinder icon), which feeds into **Embed Features to Model** (represented by a rectangle icon).
 - Model Training:** The Embed Features to Model feeds into the **NCF Model** (represented by a rectangle icon).
 - Feature Store:** The NCF Model feeds into the **Build Richer Features Store** (represented by a rectangle icon).
 - Ranking Model Training:** The Build Richer Features Store feeds into the **Train Ranking Model** (represented by a rectangle icon).
 - Ranking Model:** The Train Ranking Model feeds into the **XGBoost** model (represented by a rectangle icon).
- Integration and Flow:**
- A horizontal line separates the Retrieval and Ranking stages.
 - A dashed line connects the **Build Richer Features Store** (Offline) to the **Add feature to Candidates** (Online) step.
 - A dashed line connects the **Rank Top K Candidates** (Online) step to the **XGBoost** (Offline) model.

- Star (Level 0, Exploration)
- Fellow (Level 1, Experience)
- Fork (Level 2, Communication)
- Issue (Level 3, Cooperation)
- Pull Request (Level 4, Cooperation)
- Pull Request Comment (Level 5, Cooperation)
- Commit (Level 6, Cooperation)

CLUSTERING METHODOLOGIES FOR PROJECT GROUPING

Part 1: Unsupervised Clustering via TF-IDF and Dimensionality Reduction

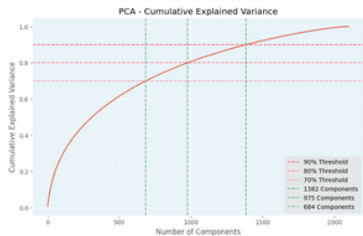
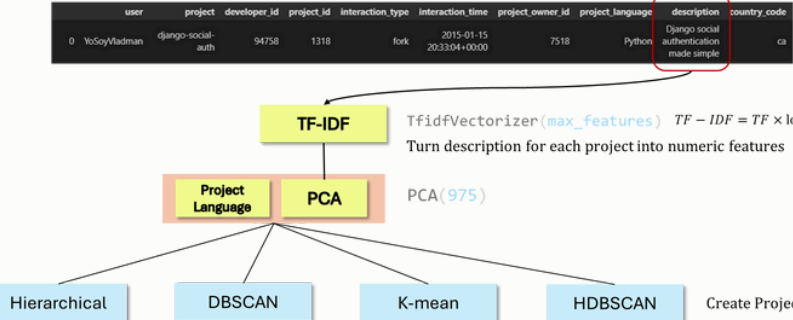


Figure 5.1: Cumulative Variance Retention in PCA at Key Thresholds (90%, 80%, 70%)

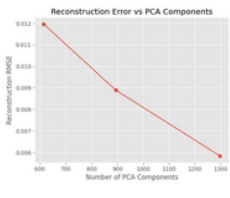


Figure 5.2: RMSE Trend Across Principal Components

Part 2: Semantic Clustering via Pre-trained Language Models: BERT

The second pipeline leverages a deep contextual language model, BERT, to generate semantically rich embeddings of project descriptions. We adopt a feature-based strategy, using the bert-base-uncased variant, which is trained on lowercase English text without fine-tuning its weights.

Part 3: Topic Modeling for Project Grouping via BERTopic

- Custom BERTopic Pipeline: The all-MiniLM-L6-v2 model. We experiment with different minimal numbers of samples in the cluster: 5, 15, and 30, respectively, and set the number of neighbors in UMAP to 15.
- Default BERTopic Configuration: The distiluse-base-multilingual-cased-v2 model.

CLUSTERING AND PERFORMANCE EVALUATION

K-Means Clustering

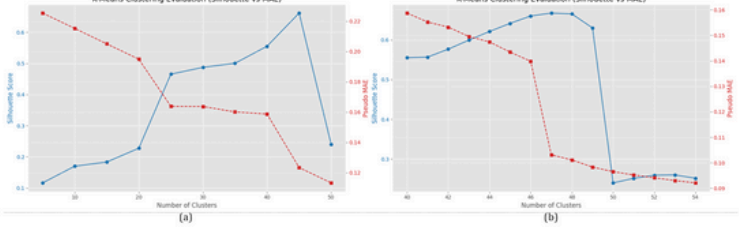


Figure 5.3: Silhouette Scores for k-means Clustering with TF-IDF tokenized project descriptions

Hierarchical Clustering

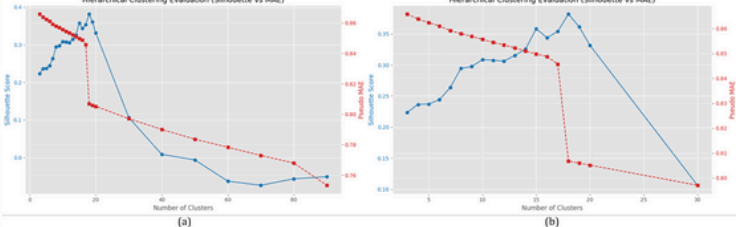


Figure 5.5: Silhouette Scores for Hierarchical Clustering with TF-IDF tokenized project descriptions

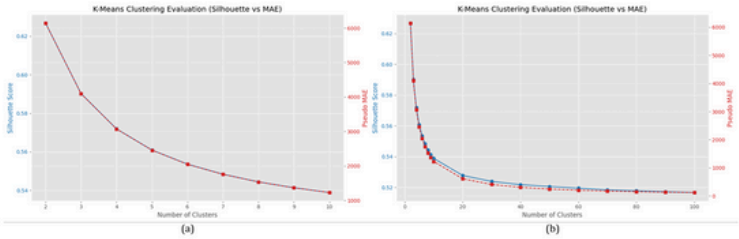


Figure 5.4: Silhouette Scores for k-means Clustering with BERT-Based tokenized project descriptions

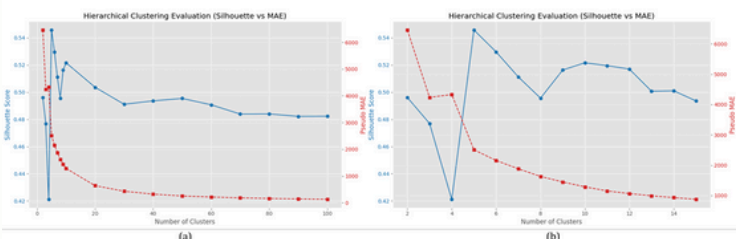


Figure 5.6: Silhouette Scores for Hierarchical Clustering with BERT-Based tokenized project descriptions

DBSCAN

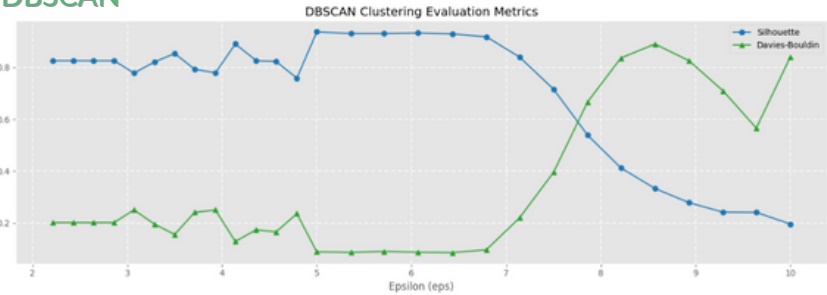


Figure 5.8: Silhouette Scores and Davies-Bouldin Score for DBSCAN Clustering with BERT-Based tokenized project descriptions

HDBSCAN

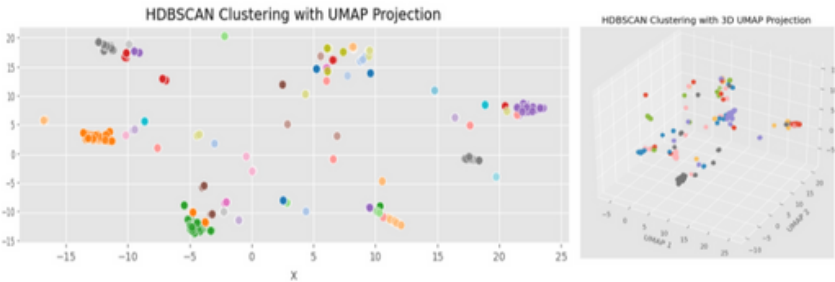
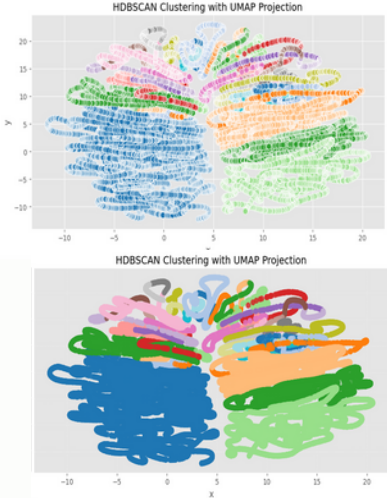


Figure 5.9: HDBSCAN as the clustering method with TF-IDF tokenized project descriptions



- HDBSCAN clustering results using UMAP projection of BERT-based tokenized project descriptions. A total of 46 clusters were identified. The left plot displays data points with white edges, while the right plot omits edges for a cleaner contrast.

TF-IDF + PCA	#Cluster	Davies-Bouldin Index	Calinski-Harabasz Score	Silhouette Score
K-means	47	0.9690	28.2601	0.6474
K-means	522	1.4632	15.2942	0.2294
Hierarchical Clustering	18	1.2344	18.5077	0.3819
DBSCAN (Eps = 0.786)	5	13.7897	16.1000	0.0236
HDBSCAN	52	2.1008	9.0208	0.1808

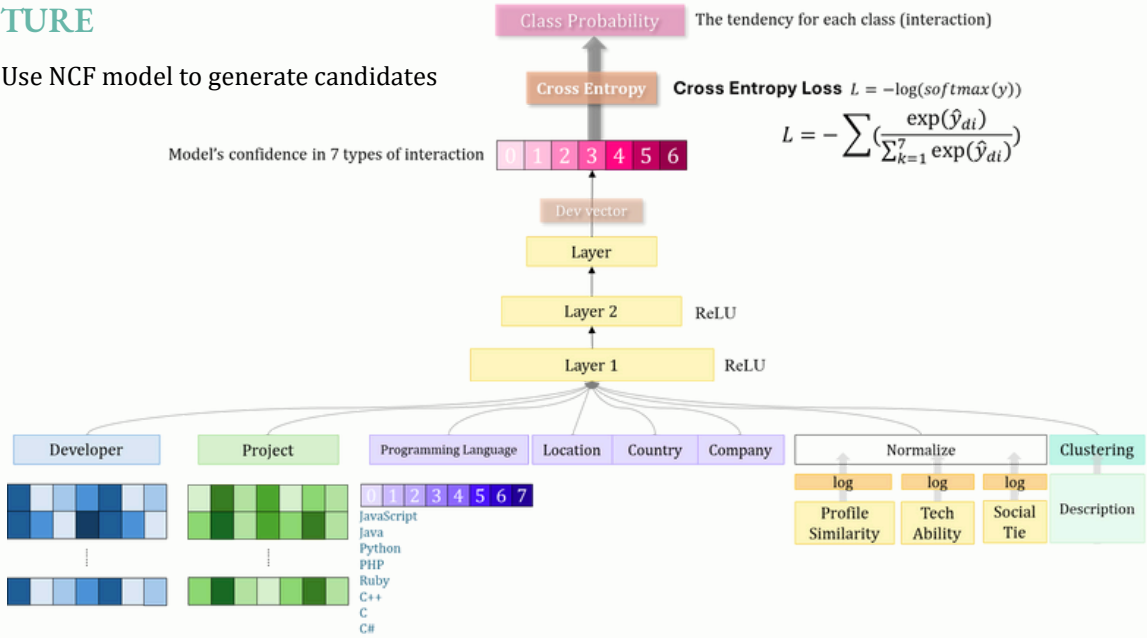
Table 5.1: Comparison of Clustering Algorithms Using Unsupervised Metrics with TF-IDF and PCA for Topic Modeling

BERT	#Cluster	Davies-Bouldin Index	Calinski-Harabasz Score	Silhouette Score
K-means	2	0.5000	147367.52	0.6267
Hierarchical Clustering	5	0.5000	276131.16	0.5457
DBSCAN (Eps = 5.00)	8	0.0883	51361618.38	0.9369
HDBSCAN	46	0.4849	14422.10	0.3269

Table 5.2: Comparison of Clustering Algorithms Using Unsupervised Metrics on BERT-Based tokenized project descriptions

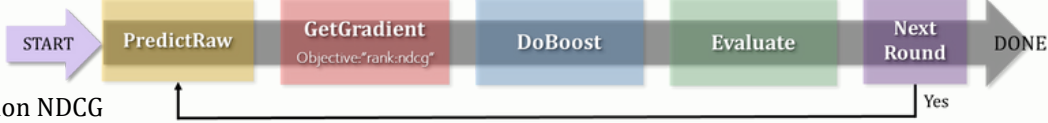
MODEL ARCHITECTURE

Candidate Generation Use NCF model to generate candidates



Learning-to-Rank (LETOR) with XGBoost

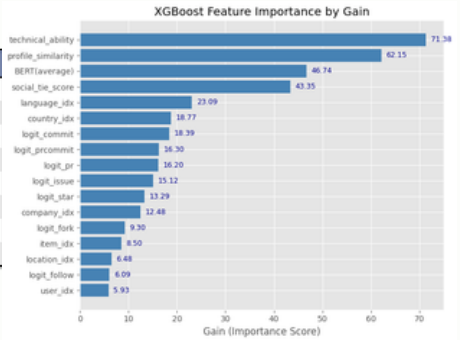
LambdaMART with objective function NDCG



RESULT & EVALUATION

Model		Recall			
		@10	@50	@100	@300
NCF Model	TFIDF-Kmeans	0.02931	0.05130	0.14657	0.31391
	BERT-DBSCAN	0.08622	0.17364	0.30035	0.45033
	BERTopic	0.01639	0.03037	0.08262	0.18262
	XGBoost	0.01834	0.02463	0.07556	0.11986
	Developer-based kNN	0.00957	0.01082	0.06866	0.09644
	Project-Based kNN	0.01832	0.02769	0.08146	0.10592
	Random	0.00036	0.00051	0.00086	0.00111
	Top 10 Popular	0.00006	0.00018	0.00033	0.00034

Model		NDCG @10	NDCG @ 50
NCF + XGBoost	TFIDF-Kmeans	0.06294	0.08522
	BERT-DBSCAN	0.08003	0.12304
	BERTopic	0.04982	0.05331
	XGBoost	0.02447	0.02740
Developer-based kNN		0.01933	0.02469
Project-Based kNN		0.01486	0.01744
Random		0.00067	0.00082
Top 10 Popular		0.00000	0.00000



CONCLUSION

- Developed a two-stage recommendation system for GitHub OSS projects using GHTorrent data (2010–2015), combining candidate generation (NCF) and ranking (XGBoost LambdaMART).
- Engineered diverse feature sets, including developer metadata, social ties, technical ability metrics, and semantic topic features from project descriptions.
- Improved semantic representation learning by systematically evaluating embedding and clustering methods; found BERT embeddings + DBSCAN clustering delivered best cluster quality (Silhouette Score 0.9384).
- Achieved significant performance gains: +111% NDCG@10 and +145% Recall@10 over NCF-only baseline, demonstrating the effectiveness of hybrid deep learning + ranking models.
- Conducted feature importance analysis, revealing that social ties, developer proficiency, and profile similarity were the strongest predictors of relevant project recommendations.
- Delivered a hybrid recommendation framework that integrates deep contextual embeddings, collaborative filtering, and learning-to-rank, enabling scalable and personalized project discovery for developers.